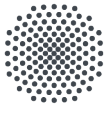
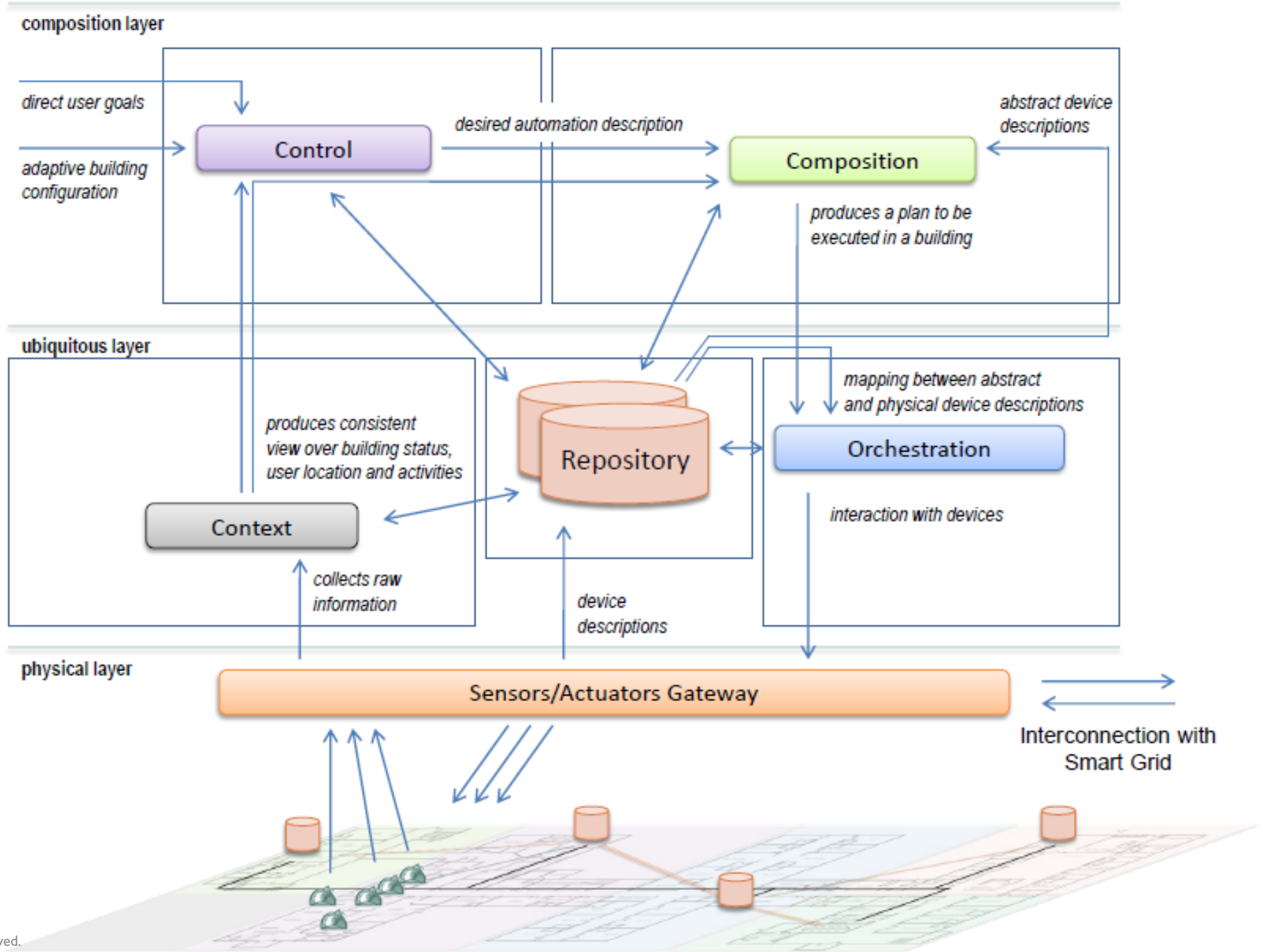


University of Stuttgart
Germany

Indirect communication

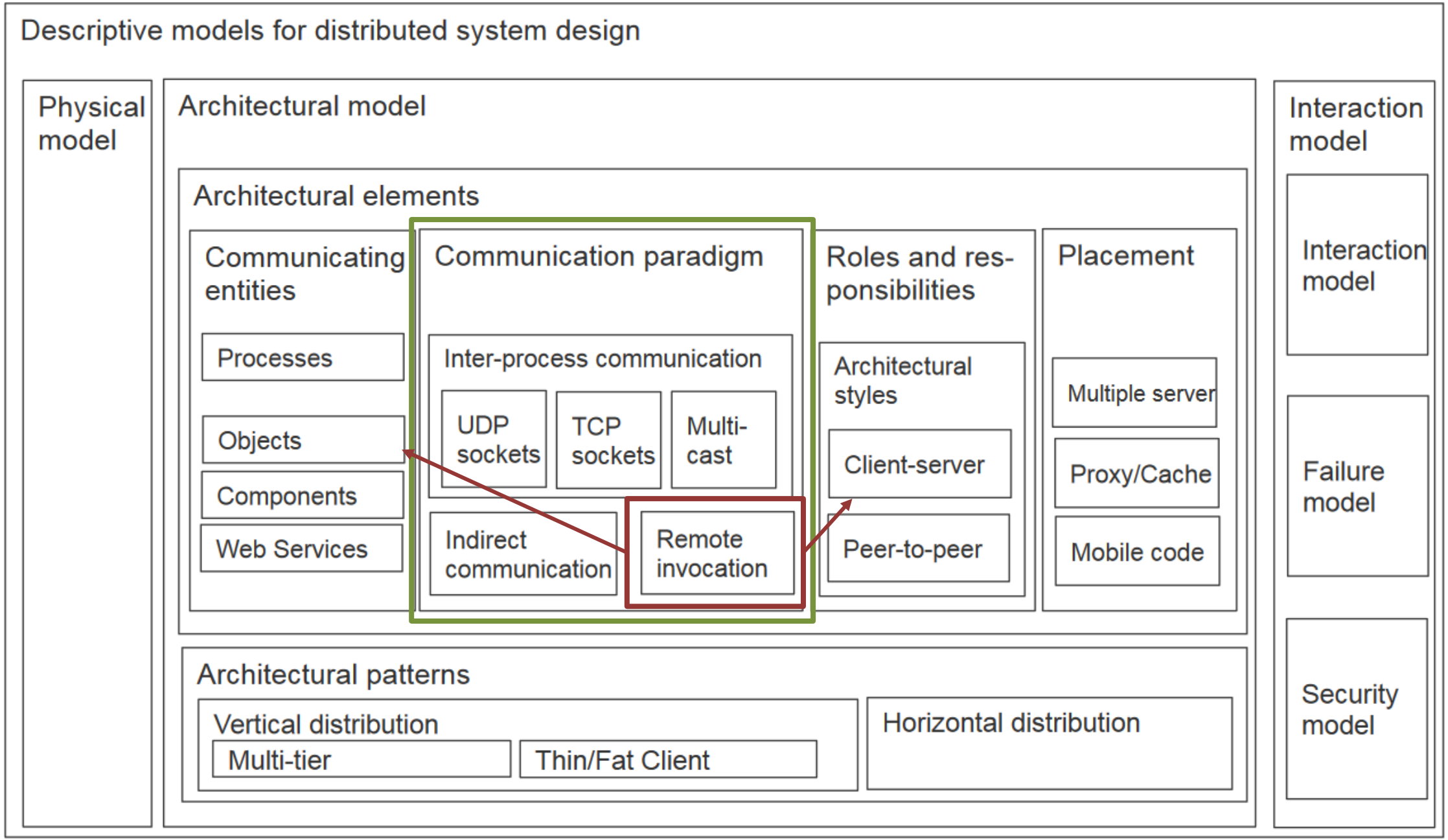
MQTT and AMQP

Ilche Georgievski

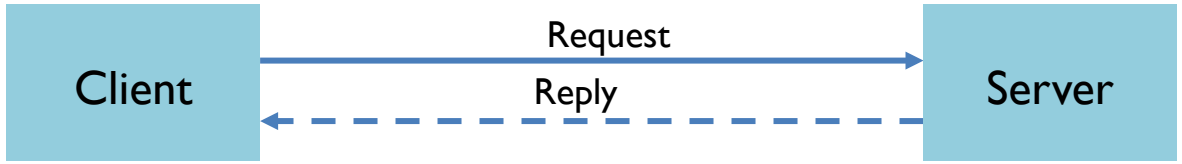


Properties of smart systems

- Get informed of events whenever they happen
- Send messages from one to many
- Near real-time delivery of messages
- Distribute minimal packets of data in huge volumes
- Minimise volume (cost) of data being transmitted
- Send messages over unreliable networks
- Reliable delivery over fragile connections
- Minimise power consumption
- Scalability



Request-reply communication



Communication initiated by client

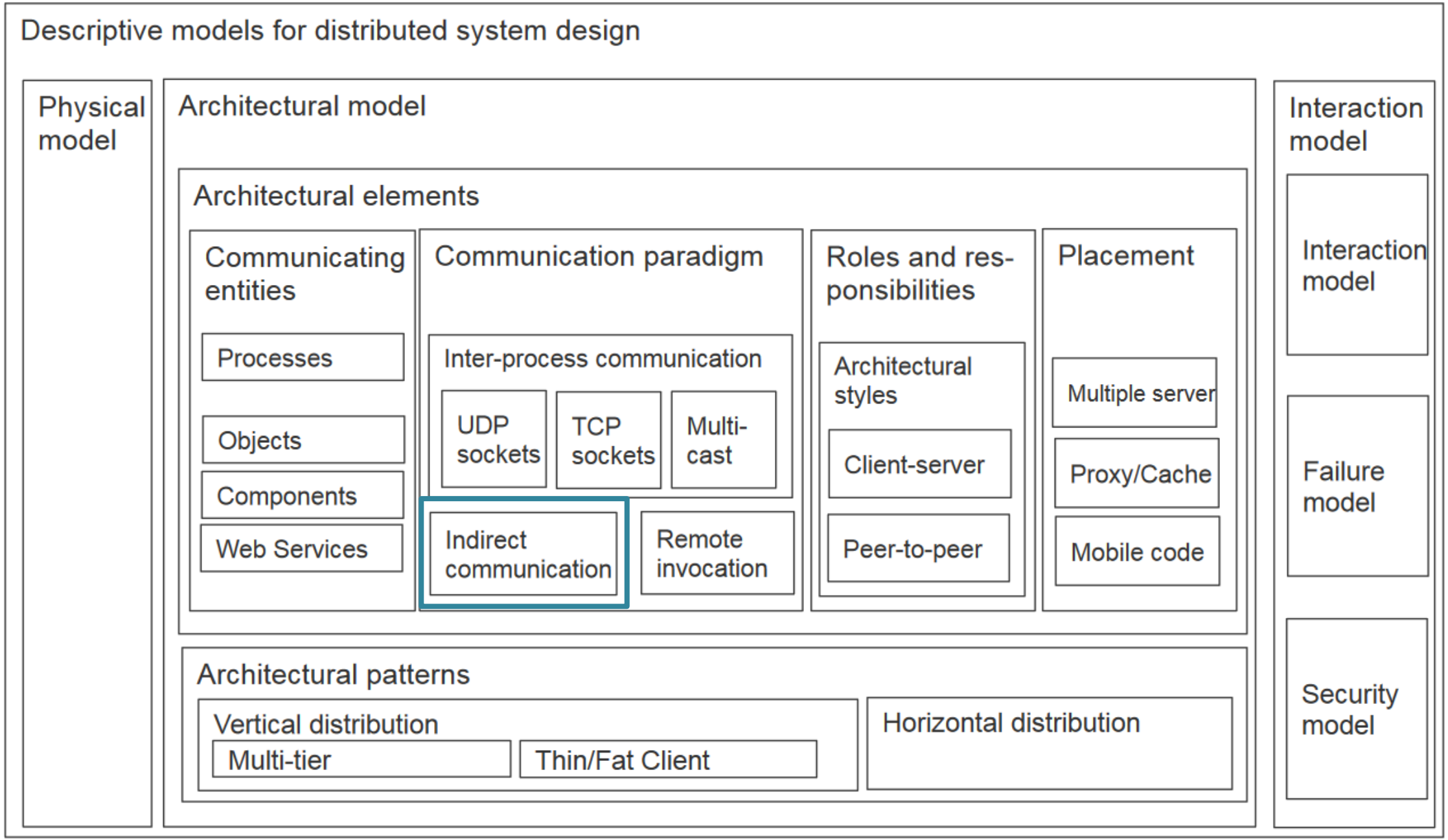
Client must know server‘s location

Synchronous communication

Messages get lost

Spikes in server load

Server must be up and running, ready for requests



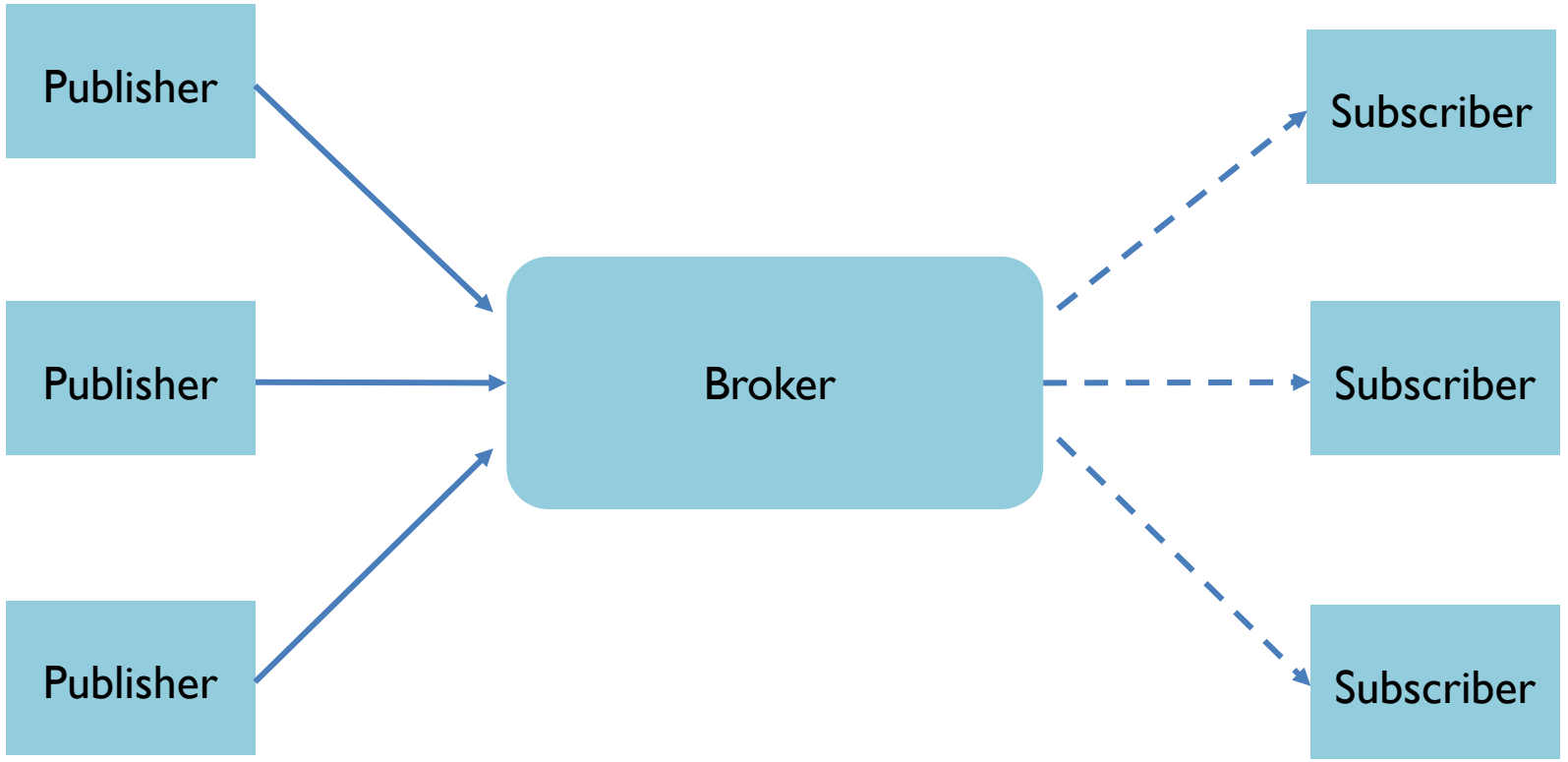
Indirect communication

	Time-coupled	*Time-uncoupled*
Space coupling	*Properties*: Communication directed towards a given receiver or receivers; receiver(s) must exist at that moment in time *Examples*: Message passing, remote invocation (see Chapters 4 and 5)	*Properties*: Communication directed towards a given receiver or receivers; sender(s) and receiver(s) can have independent lifetimes *Examples*: See Exercise 6.3
Space uncoupling	*Properties*: Sender does not need to know the identity of the receiver(s); receiver(s) must exist at that moment in time *Examples*: IP multicast (see Chapter 4)	*Properties*: Sender does not need to know the identity of the receiver(s); sender(s) and receiver(s) can have independent lifetimes *Examples*: Most indirect communication paradigms covered in this chapter

Indirect communication

- Group communication
- Publish-subscribe mechanism
- Message queues

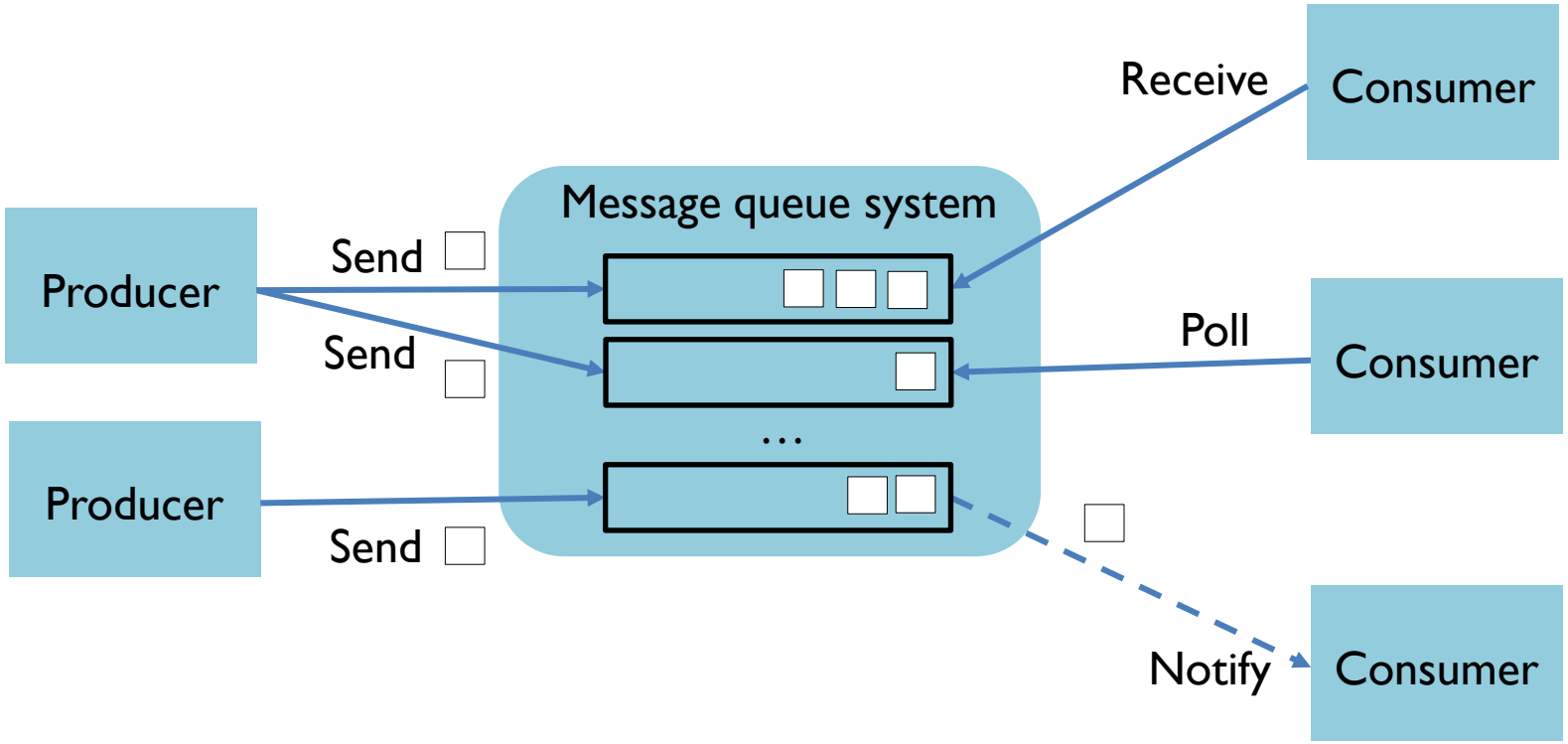
Publish-subscribe mechanism



Implementation

- Protocols
  - MQTT
  - Data Distribution Service (DDS)
  - Extensible Messaging and Presence Protocol (XMPP)
- Messaging middleware
  - Java Messaging Service (JMS)
  - ZeroMQ
  - Apache Kafka
  - RabbitMQ
  - Redis

Message queues



Implementation

- Protocol
  - AMQP
- Messaging middleware
  - JMS
  - IBM WebSphereMQ
  - ZeroMQ
  - RabbitMQ
  - Apache Qpid

MQ Telemetry Protocol

MQTT

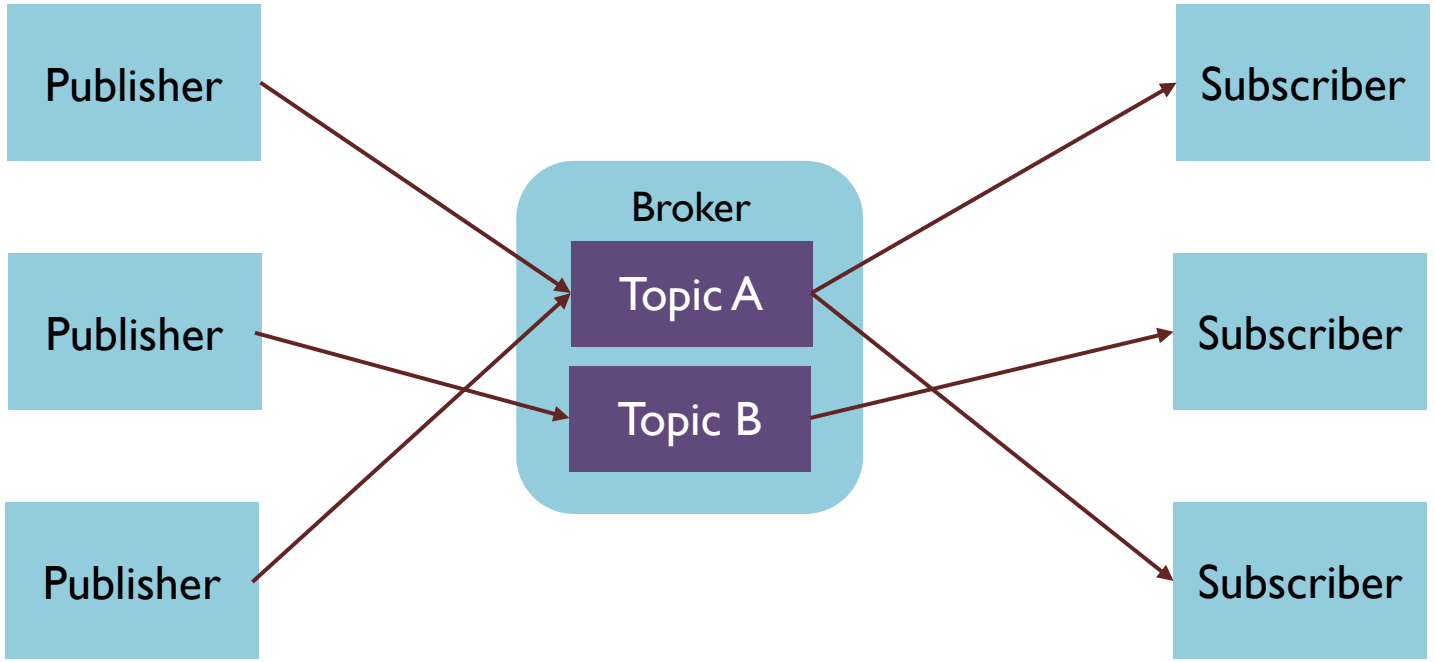
MQTT

A lightweight event and message-oriented protocol allowing devices to asynchronously communicate efficiently across constrained networks to remote systems

Intro

- Introduced in 1999
- OASIS standard in 2014 (v3.1.1)
- OASIS standard in 2019 (v5.0)
- It uses TCP as a transport protocol
- Binary protocol that normally requires a fixed header of 2 bytes with small message payloads of up to 256MB
- It relies on TLS/SSL for security

Model



Features

- 3 QoS levels
- Retained messages
- Topics
- Last will and testament
- Heartbeats

Quality of Service

- **0**: at most once
  - No acknowledgement required
- **1**: at least once
  - A message is sent multiple times until an acknowledgement is received
- **2**: exactly once
  - Sender and receiver use a two-level handshake to ensure only one copy of the message is delivered

Retained messages

- Publisher can mark a message as retained
- Broker saves the last known good message of a retained topic
- Broker gives the retained message to new subscribers
  - A new subscriber does not have to wait for a publisher to publish a message to receive its first message

Last will and testament

- Client can define an LWT connection
- After the client ‘dies’, the broker sends the message on behalf of the client

Last will and testament

CONNECT

MQTT packet	Example
clientId	“client1”
cleanSession	true
username	“ilche”
password	“mypassword”
lastWillTopic	“u38/0/353/window/temperature”
lastWillQoS	2
lastWillMessage	“unexpected exit”
keepAlive	60

Topics (1)

- A topic has a hierarchical form where “subtopic” is separated by /
- `<subtopic>/<subtopic>/…/<subtopic>`
  - `<building>/<floor>/<room>/<area>/<type-id>/<instance-id>`
  - `u38/0/353/window/temperature/12345`

Topics (2)

- Wildcards
  - For single level, use '+' anywhere in the string
    - Subscribe to all temperature readings in all offices and areas of the ground floor: `u38/0/+/+/temperature/+`
  - For multi level, use "#" at the end of the string
    - Subscribe to all readings in all rooms and areas of the ground floor: `u38/0/#`

Publishing to a topic

PUBLISH

MQTT packet	Example
packetId	513
topicName	“u38/0/353/window/temperature”
qos	1
retainFlag	false
payload	32.5
dupFlag	false

Examples

MQTT CLIENTS IN PYTHON

Temperature publisher

```
import time
from datetime import datetime
import json
from simulatediot import TemperatureSensor
import paho.mqtt.client as mqtt


class Simulator:
    def __init__(self, interval):
        self.interval = interval

    def start(self):
        ts = TemperatureSensor(20, 10, 16, 35)

        mqtt_publisher = mqtt.Client('Temperature publisher')
        mqtt_publisher.connect('127.0.0.1', 1883, 70)
        mqtt_publisher.loop_start()

        while True:
            dt = datetime.now().strftime("%d-%m-%YT%H:%M:%S")
            message = {
                "type-id": "de.uni-stuttgart.iaas.sc." + ts.sensor_type,
                "instance-id": ts.instance_id,
                "timestamp": dt,
                "value": {
                    ts.unit: ts.sense()
                }
            }
            jmsg = json.dumps(message, indent=4)
            mqtt_publisher.publish('u38/0/353/window/' + ts.sensor_type + '/' + ts.instance_id, jmsg, 2)
            time.sleep(self.interval)


s = Simulator(5)
s.start()
```

Temperature subscriber

```
import paho.mqtt.client as mqtt
import json


def on_message(client, userdata, message):
    print('Message topic {}'.format(message.topic))
    print('Message payload:')
    print(json.loads(message.payload.decode()))


mqtt_subscriber = mqtt.Client('Temperature subscriber')
mqtt_subscriber.on_message = on_message
mqtt_subscriber.on_connect = on_connect

mqtt_subscriber.connect('127.0.0.1', 1883, 70)
mqtt_subscriber.subscribe('u38/0/353/+/temperature/+', qos=2)

mqtt_subscriber.loop_forever()
```

Advanced Message Queuing Protocol

AMQP VIEWED THROUGH RABBITMQ

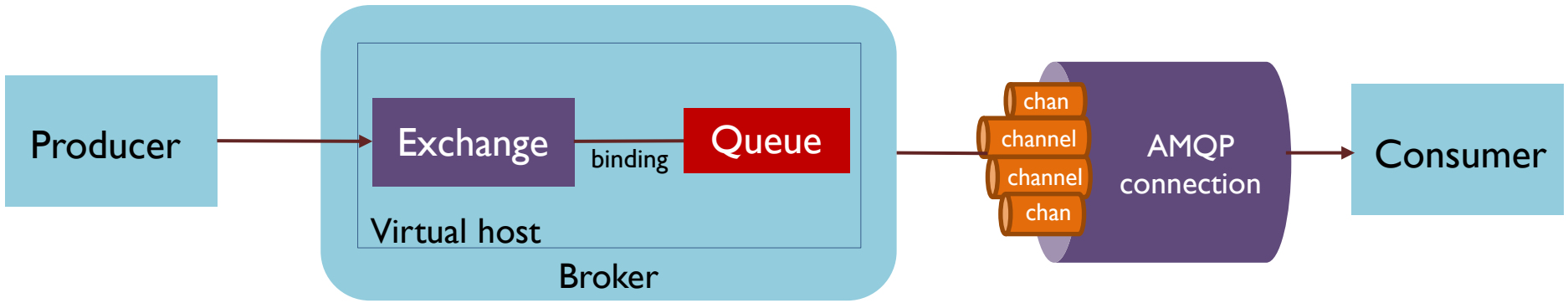
Intro

- Introduced in 2003
- OASIS standard in 2012 (v0.9.1)
- It uses TCP as a default transport protocol
- Binary protocol that requires a fixed header of 8 bytes with small messages whose maximum size depends on the broker/programming technology
- It uses TLS/SSL and SASL for security

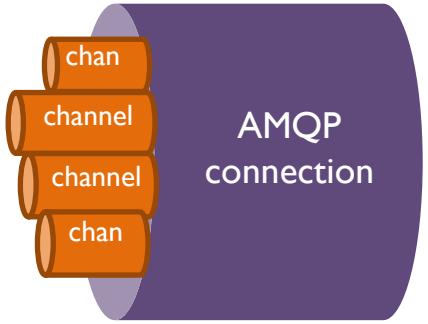
RabbitMQ

- Erlang implementation of AMQP
- Supports
  - AMQP v0.9.1
  - AMQP v1.0
  - MQTT
  - STOMP

Model



Channels

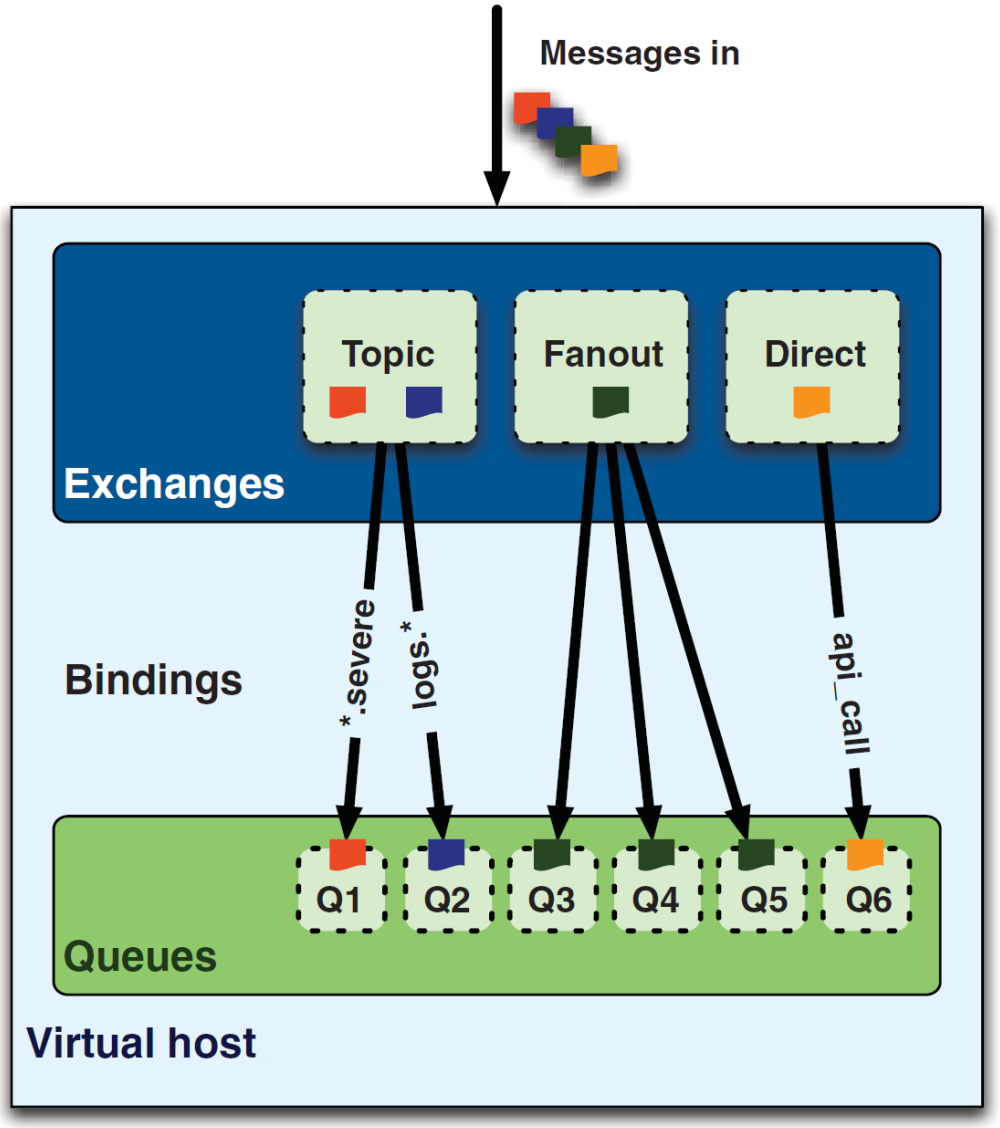


- AMQP connection is a TCP connection
- A channel is a logical connection
  - Commands are sent via channels
  - All channels reuse the same TCP connection
  - Hundreds of thousands of new channels a second

RBMQ message

- Payload
  - Anything
- Label
  - Describes the payload
  - Information about where the message should go
    - Exchange name
    - Topic/routing key (optional)

Broker



Queue

- Place for messages ready to be consumed
- How to create a queue?
  - `queue.declare`
  - Queue name
    - Used by consumers to subscribe to the queue
    - Used when creating a binding
    - Otherwise, RBMQ assigns a random name
  - Properties
    - `durable`
    - `exclusive`
    - `auto-delete`

Queue

- Who can create a queue?
  - Both consumers and producers
  - Consumers can't create a queue while subscribed to another one on the same channel
- What if a queue already exists when trying to declare it?
  - Idempotent if the properties are exactly the same

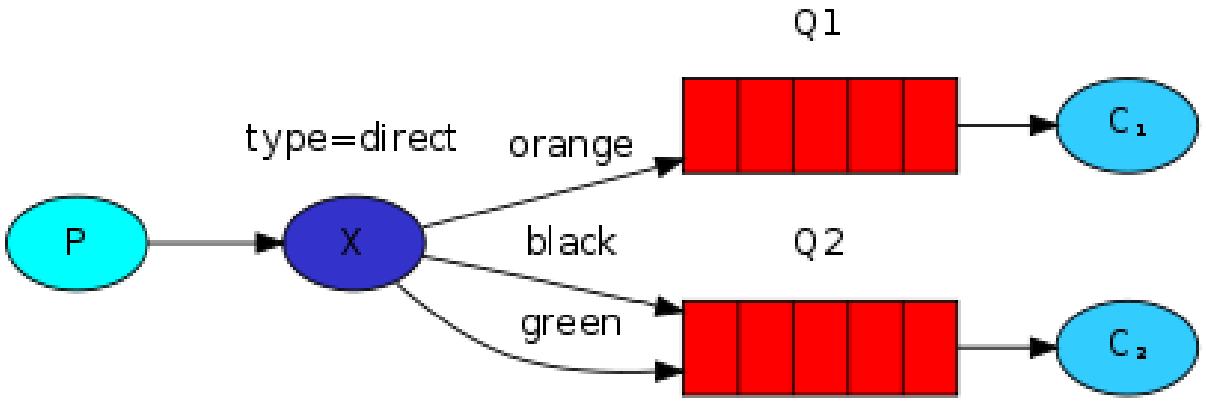
Queue and consumers

- Messages are sent immediately to consumers subscribed to the queue
  - `basic.consume`
  - `basic.get`
- Consumer acknowledges a message iff it is done with processing (to RBMQ and not to producer!)
  - `basic.ack`
  - `auto_ack` parameter set to true when subscribing to a queue (acceptable risk of losing messages)
- Reject a message (rather than acknowledge it)
  - `basic.reject`

Exchanges and bindings

- Exchange
  - Place where producers send messages
- Routing key
  - Rules upon which RBMQ decides to which queue it should deliver the message
- Binding
  - A queue is said to be *bound* to an exchange by a routing key

Direct exchange



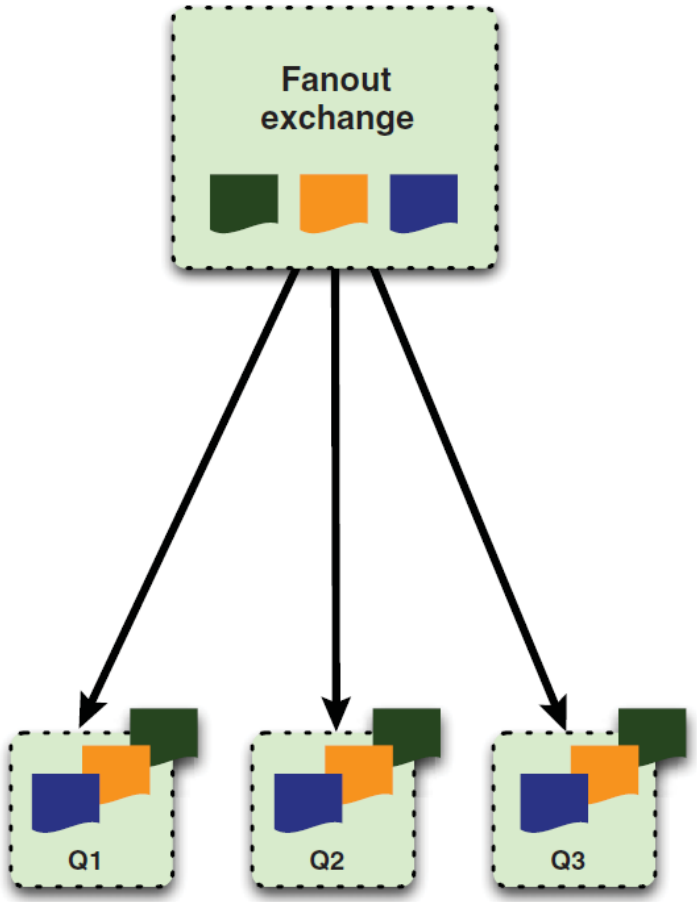
- Routing algorithm: if the routing key of a message matches the routing key of a queue, then the message is delivered to that queue
- If `Q2` is bound to `X` by the routing key `black`, then all messages with the routing key `black` will be delivered to `Q2`

Direct exchange

- Default exchange
- Empty string as its name
- A newly declared queue will be automatically bound to this exchange using the queue name as routing key

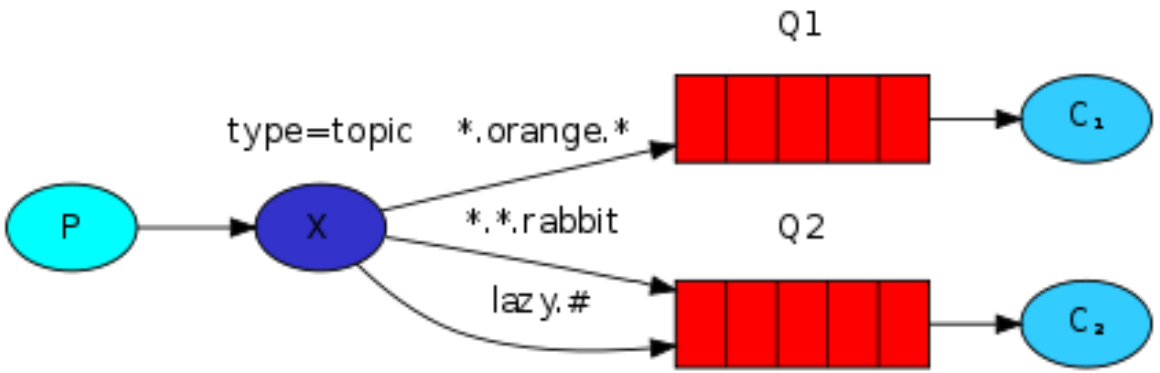
- `channel.basic_publish(exchange='', routing_key='queue-name', body=msg)`

Fanout exchange



- Routing algorithm: a message is delivered to all queues bound to the exchange
- Routing keys are ignored

Topic exchange



- Routing algorithm: if the routing key of a message matches the routing key of one or more queues, then the message is delivered to all the queues

- Routing key must be a list of words delimited by dots
  - * can substitute for exactly one word
  - # can substitute for zero or more words

Example

- `<building>.<floor>.<room>.<area>.<type-id>.<instance-id>`
- u38.0.353.window.temperature.12345

- Exchange
  - Type: `topic`
  - Name: `sciot.topic`
- Queue
  - Name: `sciot.temperature` and/or `sciot.all`
  - Type: `durable`
- Routing key: `u38.0.353.*.temperature.*` and/or `u38.#`

Virtual host

- Logical subdivision of RBMQ´s execution space
  - Virtual message broker
  - Its own queues, exchanges and bindings
  - Its own permissions
- Allow for multiple applications to one RBMQ server
- Vhosts are to RBMQ what VMs are to physical servers
- Default vhost called /

Reliability: durability

- Queues and exchanges together with messages don't survive a reboot of the RBM server
- The `durable` property of queues and exchanges is set to false by default
- For messages to survive reboot, queues and exchanges must be durable, but this is not sufficient
  - Messages must be persistent
  - Set the `delivery-mode` option of the message to 2

Example

RABBITMQ CLIENTS IN PYTHON

Temperature producer

```
import grovepi
import time
import pika

port = 7
sensor = 0
timeout = 1

credentials = pika.PlainCredentials('sciot', 'Us47*nHgD')
connection = pika.BlockingConnection(pika.ConnectionParameters('192.168.2.107', 5672, '/', credentials))
channel = connection.channel()

channel.exchange_declare(exchange='sciot.topic', exchange_type='topic', durable=True, auto_delete=False)

while True:
    [temperature, humidity] = grovepi.dht(port, sensor)
    message = time.ctime() + ' Temperature = ' + str(temperature) + ' C, humidity = ' + str(humidity) + ' %'
    channel.basic_publish(exchange='sciot.topic', routing_key='u38.0.353.window.temperature.12345',
body=message)
    print('Sent ' + message)
    time.sleep(timeout)
```

Temperature consumer

```
import pika

credentials = pika.PlainCredentials('sciot', 'Us47*nHgD')
connection = pika.BlockingConnection(pika.ConnectionParameters('192.168.2.107', 5672, '/', credentials))
channel = connection.channel()

channel.exchange_declare(exchange='sciot.topic', exchange_type='topic', durable=True, auto_delete=False)

channel.queue_declare(queue='sciot.temperature')

channel.queue_bind(queue='sciot.temperature', exchange='sciot.topic', routing_key='u38.0.353.*.temperature.*')


def callback(ch, method, properties, body):
    print('Received: {}'.format(body))


channel.basic_consume(queue='sciot.temperature', on_message_callback=callback, auto_ack=True)
print('Waiting for messages')
channel.start_consuming()
```

RabbitMQ™ 3.8.3 Erlang 22.3

Refreshed 2020-05-12 12:26:07 Refresh every 5 seconds

Virtual host All

Cluster **rabbit@ILCHE-WORK**

User **guest** Log out

Overview | **Connections** | **Channels** | **Exchanges** | **Queues** | **Admin**

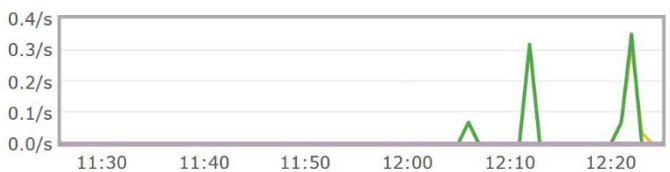
Overview

Totals

Queued messages last hour ?

Ready	0
Unacked	0
Total	0

Message rates last hour ?



Publish	0.00/s	Deliver (auto ack)	0.00/s	Get (manual ack)	0.00/s	Unroutable (return)	0.00/s
Publisher confirm	0.00/s	Consumer ack	0.00/s	Get (auto ack)	0.00/s	Unroutable (drop)	0.00/s
Deliver (manual ack)	0.00/s	Redelivered	0.00/s	Get (empty)	0.00/s	Disk read	0.00/s
						Disk write	0.00/s

Global counts ?

Connections: **0** | Channels: **0** | Exchanges: **8** | Queues: **2** | Consumers: **0**

Nodes

Name	File descriptors ?	Socket descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Info	Reset stats	+/-
rabbit@ILCHE-WORK	0 65536 available	0 58893 available	459 1048576 available	118MiB 13GiB high watermark	762GiB 48MiB low watermark	18h 57m	basic disc 1 rss	This node All nodes	

References

- Coulouris, G., Dollimore, J., Kindberg, T., Blair, G., Distributed Systems: Concepts and Designs. Fifth Edition.
- Videla, A. and Williams, J. W. RabbitMQ in Action: Distributed messaging for everyone. Manning Publications Co. 2012.

Icons designed by Freepik.